

um-game-playing-strength-of-mcts-variants

1st Place Solution

Rank	1
Team	James Day
Author	James Day
Topic ID	549801
Version	Original

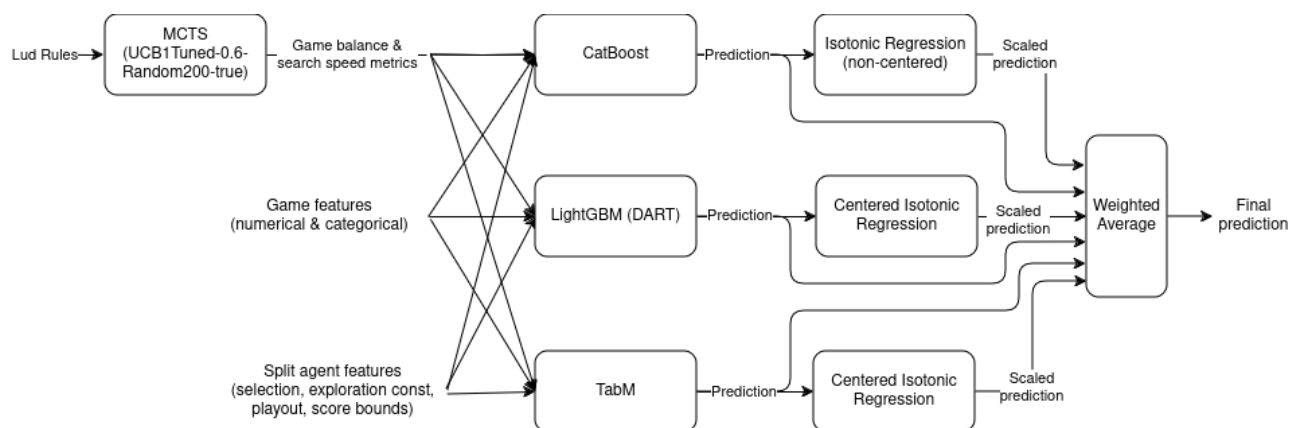
Source: <https://www.kaggle.com/competitions/um-game-playing-strength-of-mcts-variants/discussion/549801>

Retrieved with Kaggle CLI on 2026-07-03.

First, I'd like to thank Kaggle and Maastricht University for hosting such an interesting competition - I learned a lot. I'd also like to congratulate everyone who finished near the top of the leaderboard. You guys had me sweating until the end!

Overview

I believe the main trick which allowed me to win was running a tree search for a few seconds on the starting position of each game to compute additional features describing how balanced it is and how quickly the search can be executed (semi-redundant with the `AdvantageP1`, `MovesPerSecond`, and `PlayoutsPerSecond` columns provided by the competition organizer). The supplemental features (and all of the ones most other competitors were using) were then fed to a stacked ensemble of CatBoost, LightGBM, TabM, and isotonic regression models. A diagram illustrating this is included below.



RMSE scores for each of the GBDT/NN + isotonic model stacks are included below.

Base model	CV	Public LB	Private LB
CatBoost	0.362	0.415	0.421
LightGBM	0.357	0.419	0.424
TabM	0.352	0.420	0.424
All (ensemble)	0.344	0.413	0.417

Ensemble weights were tuned to my CV using Nelder-Mead.

As you might be able to tell from the table above, my local cross validation was somewhat flakey, but the ensemble which scored best in CV happened to also have the best LB scores (both public & private).

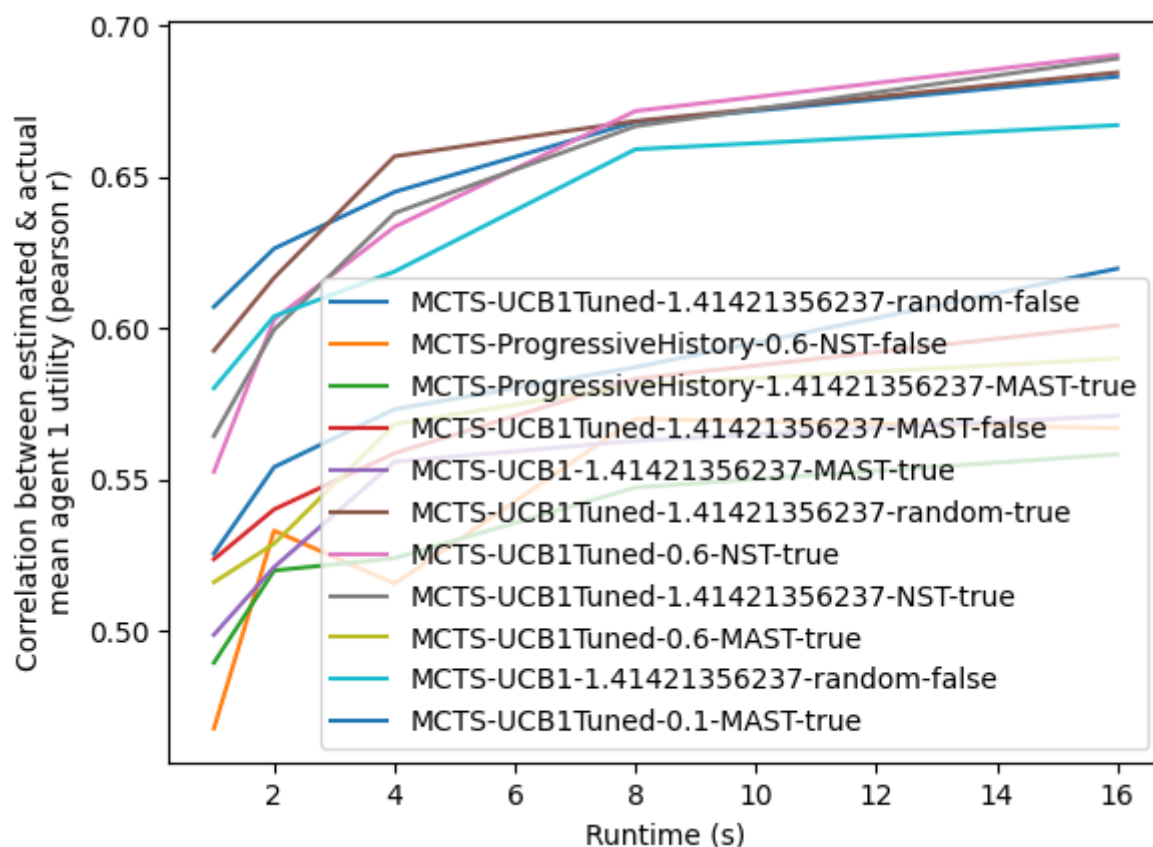
Starting position evals

The `AdvantageP1` column is computed based on play between agents which make random moves, so it is not perfectly correlated with how much of an advantage player 1 has during non-random play. Computing how much of an advantage player 1 really has by simulating games between non-random agents is too CPU

intensive to submit and training a language model to predict how much of an advantage (or disadvantage) player 1 has based on the rules did not yield very accurate results, so I instead generated a supplemental indication of how balanced each game was by running a tree search briefly on the game's starting position. I believe this yields results of similar accuracy to fully simulating games between non-random agents ~250x faster. Much like simulating the games there is a configurable tradeoff between speed and accuracy, with a 15 second search being roughly as good as 1 hour of simulating matches between non-random agents.

In early experiments, feeding models supplemental game balance metrics computed by running tree searches on the starting positions for 15 seconds per game **improved my scores by ~0.045 CV, ~0.012 LB**. This is what initially got me to the #1 spot on the public LB roughly a month into the competition.

The tree search algorithm I used to compute the game balance metrics is equivalent to the `UCB1Tuned-0.6-Random200-true` algorithm the competition organizer used to play some of the games (my code links against the Ludii player's MCTS implementation as a dependency). I believe the most important aspects of that configuration for this workload are the selection and playout values, `explration_const` and `score_bounds` didn't seem to matter. A data visualization from one of my early experiments which roughly illustrates this is included below. As you can see, the configurations which worked "well" all either used a UCB1 or UCB1Tuned selection strategy and random or NST playouts. That being said, NST seemed to work worse on the leaderboard data than the training data the competition organizer provided and UCB1Tuned seemed to work marginally better than UCB1 (in CV - not sure about LB), so the winning submission used UCB1Tuned & random playouts.



Supplemental search speed metrics

There's some random noise in the `MovesPerSecond` and `PlayoutsPerSecond` columns provided by the competition organizer. As a result, while computing the starting position eval/game balance metrics described above, my code also reports information about how quickly it was able to explore the game tree, namely the number of actions & search iterations per second. Feeding these features to my models improved my CV scores by ~ 0.005 , so they were used in the winning solution, but they did not seem to have any significant impact on the LB scores (the impact was within random run-to-run variance).

Additional training data

I generated a total of 14,365 extra training rows with 484 extra unique rulesets. The extra rulesets were generated via a mixture of two approaches:

1. Implementing & running my own version of [GAVEL](#).
2. Ordinary instruction-tuned LLMs (Llama 3.1 70B & Qwen 2.5 32B) with few-shot prompting.

Of the 484 annotated rulesets, 391 were produced using approach #1, 93 were produced using #2. I generated far more games with approach #2 (over 10k of them), but wound up discarding and heavily downsampling them due to quality issues.

The rulesets generated using approach #1 were generally much higher quality than #2 (at least according to the metrics GAVEL is selecting for, namely balance and strategic depth), but #1 produced "playable" games much more slowly than #2, even accounting for the fact $\sim 95\%$ of the rulesets generated using #2 had to be discarded due to Ludii compilation or runtime errors. I don't recall exact throughput or error rate statistics off the top of my head, but believe there was an order of magnitude difference in the rate at which playable rulesets were produced by each approach.

As for the data annotation portion of this, I attempted to save CPU time by having each (ordered) agent pair only play 4 matches per ruleset (e.g. 4 matches with agent "A" moving first, then another 4 with agent "B" moving first), so the my labels were pretty dirty in comparison to the ones from the competition organizer, but using the extra data still improved my scores by ~ 0.004 CV, 0.001 - 0.002 LB. I mitigated the label noise by weighting the extra training samples less heavily than the ones provided by the competition organizer. Some models seemed more sensitive to the noise than others, so I tuned the extra data weight separately for each type of model. The weights used in the winning solution are outlined below. These were tuned to my local cross validation, not the leaderboard.

Model	Extra data weight
CatBoost	0.25
LightGBM	1.0
TabM	0.5

Scaling up the amount of supplemental data seemed to produce severely diminishing returns (e.g. the first batch of ~7k extra training rows improved my CV scores by ~0.003 but the second only produced a ~0.001 improvement), with the optimal extra data weights falling as I scaled up, so I suspect label noise was becoming an increasingly large problem as I scaled up. With the benefit of hindsight, I suspect I might have gotten better results by annotating half as many games with double the compute per game.

Data augmentation

I used two forms of data augmentation:

1. All of the tree search related features for the training examples were computed 10 times. The values included in each row fed to the models during training were randomly selected from 9 runs that were not held out for cross-validation purposes, so models were strategically exposed to the random noise in the features, rather than having the same errors duplicated across all of the rows for each ruleset.
2. All of the nondeterministic features in the data provided by the competition were recomputed 5 times and used to compute a more accurate (less noisy) estimate of the "real" values of the features. The values fed to the models during training were uniformly sampled between these estimates and the ones provided by the competition or during my initial data generation runs.

1 improved both my CV & LB scores by ~0.002, with a good correlation between CV and LB. The impact of #2 was a bit murkier as its impact depended on whether or not I dropped some of the features impacted by (suspected) Ludii player version discrepancies discussed in the next section, but was generally in the ballpark of ≤ 0.002 .

Feature selection

While re-annotating the rulesets provided by the competition organizer, I noticed there were 43 features whose values were perfectly consistent from one run to the next when I generated them on my machines, but different from the values provided by the competition organizer. My best guess is that this was likely caused by some sort of Ludii player version discrepancy (I was using version 1.3.13), but I don't have super concrete evidence to prove that. Interestingly, when I tried dropping the columns with inconsistent values, my CV scores consistently improved by 0.001-0.002, which was surprising to me as no other feature selection strategy consistently resulted in a score improvement across multiple random seeds and models. It appears all of these features need to be dropped at once in order for there to be a measurable benefit (not one at a time) and importance based filtering did not do a good job identifying them as unimportant/poisonous.

The winning submission did not use any of the features impacted by the suspected version discrepancy. The LB impact of this is a bit unclear because I submitted multiple changes that seemed to help in CV at once in the last 12 hours without testing incremental versions - more on that later.

Isotonic regression

This was largely inspired by a trick I saw in some public notebooks in which people were multiplying their raw predictions by some coefficient then clipping to be in a range narrower than -1 to 1, but seemed to produce a better CV-LB correlation. More specifically, using the scale + clip trick with parameters tuned to my CV did more harm than good on the leaderboard, but fitting isotonic regression models which use OOF predictions as features and running them as a postprocessing step improved both my CV & LB scores by around 0.002.

For most base + isotonic model stacks, I used [centered isotonic regression](#), which seemed to score slightly better in cross validation than scikit-learn's "traditional" isotonic regression implementation, presumably due to the strict monotonicity constraint making CIR models less prone to overfitting. By "slightly" better, I mean a difference in the 4th decimal point. However, for CatBoost it caused some crashes due to lacking functionality analogous to scikit-learn's `out_of_bounds='clip'` parameter, which I did not bother working around properly for CatBoost (clipping raw predictions to the range -1 to 1 at inference time before feeding them to CIR doesn't help if the training examples were in a range narrower than that!), so the CatBoost model stack used scikit-learn's non-centered isotonic regression implementation.

TabM

In the last month of the competition, Yandex (the authors of CatBoost) released a new tabular deep learning library and research paper describing it, TabM (<https://github.com/yandex-research/tabm>, <https://arxiv.org/pdf/2410.24210>). Deep learning based solutions did not seem very promising in some of my experiments earlier in the competition, but the benchmark results in their paper were impressive, so I decided to give it a try. It worked surprisingly well! More specifically, it turned out to be competitive with LightGBM on the leaderboard and was my strongest single model in cross validation. During early experiments adding it to my previous-best ensemble improved LB scores by ~0.001, but I'd need to do some ablation testing to say how much it helped in the final ensemble.

Hyperparameter tuning

The winning solution's hyperparameters were selected by running Optuna a half dozen or so times per model type with 5-fold cross validation and only a single random seed, then re-checking each of the "optimal" configurations with 10-fold cross validation and 3 seeds. CV was [GroupKFoldShuffle](#) partitioned on the ruleset names.

A few things I found relatively noteworthy about the winning config are outlined below.

- TabM's piecewise linear embeddings for continuous numerical features seemed to help tremendously. I suspect this is the main reason it was able to beat a relatively traditional MLP.
- The optimal config for TabM was slightly outside the search range described in the original TabM research paper. My best results were achieved with relatively wide hidden layers, small batch sizes, and low learning rates.

- Setting LightGBM's `boosting` parameter to `dart` was critical to getting it working well enough to be useful. This improved my LightGBM-only scores by roughly 0.004 CV, 0.003 LB, but came at the expense of a 5-10x increase in training runtimes. My hyperparameter searches were run both with and without `dart`.

Ensembling

One detail which isn't clear in the overview that I figure I should probably mention is that each of the model blocks show above is actually an ensemble of 20 models, the models trained during 10-fold cross validation repeated for 2 random seeds. CatBoost & TabM rely on early stopping to work well, so the individual models can't really be trained on all the data - ensembles was the best way to make use of all of it. I generally repeated my CV runs for 3 seeds but only used models from 2 of them in order to conserve memory in the inference notebook (of the 30 GB RAM, 18 are used for tree searches and 10 are used for the models, so I can't really scale up to a third seed). The 2 seeds used were selected arbitrarily, not tuned in any particular way.

"Trust CV" vs. "Trust LB"

During the last week or so of the competition, I pursued two strategies for creating final ensembles in parallel.

1. "Trust LB" - I tried selecting candidate configurations proposed by optuna that randomly happened to work well on the leaderboard when trained with "version 4" of my extra data (~half as much as the winning solution) and tuned the ensemble weights to the leaderboard. The candidate models used while tuning this approach to the leaderboard did not make full use of the feature selection & data augmentation approaches described above because most of my hardware was focused on approach #2...
2. "Trust CV" - I tried selecting candidate model hyperparam configurations proposed by Optuna by double checking them with 10-fold CV run for multiple seeds and more thoroughly tuned the data augmentation, feature selection, and extra data weight choices to my CV. This process was done with "version 6" of my extra training dataset, the one described above. The choices made during this process were not tested on the leaderboard until the last 12 hours of the competition or late submissions after the competition ended.

Surprisingly, "Trust CV" beat "Trust LB" on both the public and private leaderboards. I really did not expect that in light of how much the public LB seemed to hate ensembles with weights tuned to my CV, how little the final 2x scaling in the amount of extra training data seemed to help (in early experiments it made my LB scores worse!), and how weakly correlated my CV scores previously seemed to be correlated with the public LB. The "Trust CV" strategy was mostly intended to avoid falling victim to a major shakeup in case the public and private leaderboards were poorly correlated with one another, I did not pursue it with the expectation that I'd be able to beat the public LB score of an ensemble tuned to the public LB.

Strategy	CV	Public LB	Private LB
"Trust LB"	0.3554	0.4142	0.4190
"Trust CV"	0.3442	0.4137	0.4178

Things I tried that didn't work very well

- Using unsplitted agent strings as categorical features
- TF-IDF & LSA
- MLPs
- XGBoost
- Various deep learning based approaches for generating text embeddings as extra features
- Training language models to predict how balanced each game is based on the lud rules
- OpenFE
- The extra features I saw people using in public notebooks
- The model stacking approach I saw people using in public notebooks (OOF predictions as features)
- Importance based feature selection
- Iterative forwards and backwards feature selection
- Training models to predict average elo ratings for MCTS-based agents in which each part of the MCTS config is set a particular way (e.g. average elo of agents which use random playouts for a particular game) and using those estimated elo ratings as supplemental features.
- Training tabular classification models to identify horrendously imbalanced and drawish games that always end a particular way (agent 1 win, agent 2 win, or draw) and incorporating the resulting predictions into an ensemble.
- Adding random noise to some of the features for data augmentation purposes
- Enforcing monotonicity constraints with CatBoost
- Using CatBoost's finetuning functionality to "improve" the predictions from other models (similar to [this](#) winning solution to a past competition)
- Using ensembles of multiple MCTS algorithms to produce the starting position evals. At best, this seems equivalent to letting a single algorithm run longer per ruleset.

I think it is plausible that some of the items above could work well with additional effort. I frequently move on when early results do not seem promising without making a super determined effort to get things working well.

Avenues for further improvement (things I missed)

I completely overlooked the flip/inversion data augmentation trick that was used by pretty much all other top teams! Looking at their writeups, I suspect this is the main reason other people were able to score almost as well as me without running tree searches on the game starting positions. If the statistics in [@goldenlock's writeup](#) are correct, then I believe a hybrid solution could likely score in the ballpark of 0.407 on the private leaderboard (0.01 better than my 1st place solution ☒).

Links

- Inference notebook
- Extra training data
- Starting position analysis, training, test, and data generation code

Supplemental Q&A

Robert Hatch · 2024-12-04T02:04:58.980000

Do you do anything besides defaults on LGB's dart-specific hyperparams?

I've tried Dart half a dozen times, and usually give up after it has trouble even converging, and i try tweaking params without obvious direction to fix the convergence issue.

James Day · 2024-12-04T02:17:07.347000

Yeah, I tuned several dart-specific parameters. My best LGBM config is included below.

That being said, I also got pretty good results with dart even before tuning any of the dart-specific parameters. A config that beat the default boosting method during earlier experiments is included below. This was generated by re-tuning all the non-dart-specific parameters, but not super thoroughly, only 20 trials.

```
    : ,
    : ,
    : ,
    : ,
    : ,
    : -
}
```

Geremie Yeo · 2024-12-04T02:30:13.743000

```
fitting isotonic regression models which use 00F predictions features them a postprocessing
step improved both CV & LB scores .
```

Nice approach for postprocessing, very creative! I also tried to do something similar by training extra models with quantile loss with the aim of increasing the absolute values of the predictions. The performance was similar to multiplying the predictions by a constant factor on both CV and LB.

```
two quantile loss models, one targeted towards the - direction (alpha=.) and the other targeted
towards the + direction (alpha=.). The initial negative predictions were ensembled with the
alpha=. model, while the initial positive predictions were ensembled with the alpha=. model, with
a weight of . for the initial predictions and . for the quantile loss prediction.
```

Have you tried anything like this in this comp, and if so, did it fare better than IsotonicRegression?

Congrats on the solo win and it is very impressive you managed to achieve so much in 3 months!

James Day · 2024-12-04T02:49:04.533000

I did not try that. Only the scale + (optionally) clipping trick used in public notebooks, isotonic regression, and centered isotonic regression.

Your writeup indicates that it worked ~as well as just scaling the predictions, which for me worked worse than scaling + clipping, and scaling + clipping with parameters tuned to the leaderboard worked roughly as well as isotonic regression did without tuning, so I have a suspicion that quantile loss model approach probably works worse than isotonic regression, but it is plausible this may largely be due to differences in how our raw predictions were distributed....

The objective of my post processing wasn't just to increase the absolute values of the predictions. When clipping, the optimal range was something like -0.96 to 0.96, and the trained isotonic regression models behave somewhat analogous to sigmoid/tanh functions, so I believe my most extreme raw predictions needed to be scaled inwards, not outwards. It is plausible you may have had success with different post-processing approaches because your raw predictions had different biases.

It is certainly an interesting idea though, thanks for sharing!

Ruby · 2024-12-04T06:59:00.587000

Thanks for your great solution but I wonder if those simulation based features against host post in this thread? I also have similar ideas at the beginning but didn't give it a try after see host post.

You are not permitted to upload Ludii itself as part of your submission and use it to run simulations directly on the test data inside your submission. This would also be infeasible to do at any meaningful scale given the time required for these simulations and the constraints imposed by the new Evaluation API. But anyway,

only that trick is not permitted.

<https://www.kaggle.com/competitions/um-game-playing-strength-of-mcts-variants/discussion/532348>

James Day · 2024-12-04T07:17:27.857000

Yeah, that worried me. I concluded that it is probably fine because:

1. I'm not using the Ludii player to run simulations.
2. I did not make any modifications to the Ludii player itself and therefore am able to distribute my solution without violating its CC BY-NC-ND 4.0 license. Older versions actually did use a modified version of Ludii, but I factored my code out of it into a separate jar file which links against it as a dependency in order to sidestep this.
3. My solution doesn't seem to violate any of the formally stated rules under the rules tab.

Part of me was tempted to make a forum post asking for clarification, but I was scared to do that because it might reveal what I was doing to other competitors, thereby causing me to lose my advantage. I also considered trying to rewrite the portions of the Ludii player my code depends on in C++ in order to eliminate any doubt regarding whether or not my solution conforms with the rules, but did not have time for that.

Mart Preusse · 2024-12-04T08:26:18.227000

Wow, congratulations! Thank you for sharing your detailed solution. This victory is well-deserved, as it clearly involved a lot of effort and creative thinking in modeling. I didn't quite understand how you estimated the true advantage with tree search, but I'll probably follow the links to get a better grasp of it.

I believe it's a good strategy not to spend too much time on ideas that don't show promise from the beginning. I wasted a lot of time on feature reduction, always thinking it might be the key to better performance. On the other hand, I dismissed data augmentation on flipped agents too early and didn't even check the cross-validation on my best CatBoost model or the public score. So, a significant part of exploiting the right strategies is having the experience to know when to quit and when to dig deeper. I'm really impressed with your strategic handling of this challenge.

James Day · 2024-12-04T12:48:47.303000

I didn't quite understand how you estimated the true advantage with tree search, but I'll probably follow the links to get a better grasp of it.

It may not be super obvious from following the current links. The relevant code is a sneaky dependency of the inference notebook. You can kinda access it by downloading `AnalyzeGameStartingPosition_v3.jar` & `Ludii-1.3.13.jar` from one of the notebook's input datasets, unpacking them, and taking a look at the code inside of them, but it is pretty mangled as a consequence of the compilation process. I plan to open source a more readable copy within the next week or two alongside all my data generation, model training, and testing code.

To elaborate a bit further on how the true advantage is estimated, when Ludii's MCTS implementations pick moves, they end up implicitly also calculating an estimated "utility" value which is essentially an indication of who is winning. My code hijacks that in order to compute a measure of how balanced the game is. If an agent thinks it is winning or losing by a substantial margin before any moves have been played, the game probably isn't very balanced.

Vasilis · 2024-12-04T19:33:46.153000

@jsday96 Congrats on the solo gold, you did so much amazing work. One question, i dont get why there are such a huge number of `n_estimators`. Isnt more than 1k trees too many? i was always fixing the number of trees to 300 or 500 when i try to optimize catboost/xgboost/lgbost

gezi · 2024-12-04T23:11:48.500000

My conclusion of 0.01 improvement using flip aug is at very early stage for my experiments before adding manual cross feats $20 * 20$, based on your high CV/LB model it might not improve that much but should still have big improve I assume .

Also @jsday96 for tabm do you use tree model output of leaf feats or oof preds as features? I wonder if it could achive high cv and LB without tree feats.

I just tested centered isotonic regression work better then simply optimize $wx+b$ about 0.0005 improvement, learn a lot from your!

James Day · 2024-12-04T23:35:04.237000

for tabm do you use tree model output of leaf feats or oof preds as features? I wonder if it could achive high cv and LB without tree feats.

It did not use any leaf features or oof predictions. Those TabM CV & LB scores were achieved without any reliance on tree models. I'd say "purely DL based" if it weren't for the post-processing.

I just tested centered isotonic regression work better then simply optimize $wx+b$ about 0.0005 improvement, learn a lot from your!

Neat! It's nice to have some confirmation that the apparent strength of CIR wasn't just due to a unique quirk of my raw predictions 😊.

James Day · 2024-12-04T23:57:12.767000

@roberthatch - LightGBM does not support using early stopping in conjunction with dart, so my best submission's LightGBM models were not trained with early stopping. I've read about workarounds for that limitation, but did not try them because I figured early stopping probably doesn't matter very much from an accuracy standpoint so long as `n_estimators` is tuned, which it was in this case.

@vasileioscharatsidis - I did my hyperparameter tuning without much regard for theoretical considerations or common rules of thumb. I picked ranges that I thought were reasonable, ran optuna, adjusted my search ranges if the best config optuna found was on the edge of what I specified (which I viewed as a sign I was wrong about the reasonable range), and conditionally repeated that search range refinement process until I felt I had ranges that covered the region in which good configurations lie decently well (with a decent bit of padding in case the results of the early optuna runs were misleading). I then ran optuna at least a couple more times with a "good" search range in order to generate additional candidate configs that are worth testing more thoroughly. The search range refinement process resulted in configs that were almost entirely dictated by empirical observations, not my initial opinions about what configs are likely to work well.

As for `n_estimators`, specifically, I think 300-500 is crazy low for this competition's dataset. My search ranges for that parameter were typically on the order of thousands to tens of thousands, not hundreds. The LGBM configs which worked well enough for me to bother recording them used 5586 - 26412. The best dart

config happened to use the biggest value optuna selected during any run, but was well below the upper bound of the search range used to produced it (35000).

Dennis Soemers · 2024-12-07T20:57:56.267000

For sure I don't mind the way in which you used it.

I see now that I wasn't as explicitly clear as I could/should have been in the post [@w5833946](#) quoted there, but really, the spirit of that rule was to prohibit running simulations in order to generate (approximations of) the target labels for the test data. That was always the single biggest concern we had during our preparation for this competition: our target labels are inherently the outcomes of computer simulations which anyone could run. This is we turned the competition into a Code competition (lets us hide test features from the competitors), this is why we had to design a whole bunch of new games for test data (couldn't just separate out some of the many games we already had because they were all public). This is also why, when the Kaggle staff informed me about the brand new API they had developed in which test data is only presented in small batches that need to be processed within relatively short amounts of time each, I agreed to use that for this competition: it effectively made it impossible for entrants to generate accurate test labels inside their submissions, as the per-batch time limit would be too low.

The processing you do with Ludii does not generate (approximations of) the labels for the test data. It's just to generate some extra features. For me, it's also key that it's a very quick search (just 15 seconds). What I'm really interested in ultimately is: when we are faced with a brand new game, can we quickly determine what the best algorithm would be to use this particular game? A quick 15 seconds search arguably fits the definition of "quickly", whereas a whole bunch of extensive simulations to actually try and evaluate all algorithms (probably taking multiple hours) wouldn't.